

RCPVMS

Reactor Coolant Pump Vibration Monitoring System

프로젝트 인수인계 문서

2026년 5월

1. 프로젝트 개요 및 목적

1.1 프로젝트 정의

항목	내용
시스템 명칭	RCPVMS (Reactor Coolant Pump Vibration Monitoring System)
형태	Electron 기반 Windows 데스크톱 애플리케이션
목적	원자력 발전소 냉각재 펌프(RCP) 진동 신호 시각화 및 AI 이상 탐지
대상 데이터	RCPVMS 전용 .BIN 바이너리 파일, DMD 계측 장비 출력 파일

1.2 핵심 기능 목록

탭	기능
양상불 분석	ResNet18 + CNN1D 양상불로 정상/이상 4-class 분류 + OOD 탐지
MAE 분석	Masked Autoencoder 기반 재구성 오차 이상 탐지
DMD 분석	DMD 파일 → RCPVMS BIN 변환 + 궤도 뷰어
RCPVMS 뷰어	BIN 파일 궤도(orbit) 그리드 시각화 (단일/배치)

1.3 모니터링 대상

- RCP 4개: RCP1A / RCP1B / RCP2A / RCP2B (NIMS 표준 명칭)
- 채널 타입: 변위(orbit, ch_type=1), 가속도(ch_type=0), Keyphasor(ch_type=2)

2. 기술 스택

2.1 Frontend / Desktop

항목	버전	내용
Electron	^38.1.2	메인 프로세스 + 브라우저 창 관리
electron-vite	^4.0.1	빌드/번들링/HMR
React	JSX	렌더러 UI
TypeScript	^5.9.2	Main/Preload 계층
better-sqlite3	^12.5.0	로컬 DB (로그, 인증)
ExcelJS	^4.4.0	결과 Excel 내보내기
electron-store	^6.0.1	앱 설정 영속화

2.2 Backend (Python 데몬)

항목	버전	내용
Python	3.10+ (3.12 권장)	가상환경: venv/
PyTorch	>=2.0.0	CPU; GPU는 CUDA 버전으로 교체
torchvision	>=0.15.0	이미지 변환
NumPy	>=1.24.0	수치 연산
Pillow	>=10.0.0	이미지 생성/처리
SciPy	>=1.10.0	신호 처리, 필터링
Matplotlib	>=3.7.0	축 레이블 렌더링

2.3 IPC 통신

- Electron ipcMain / ipcRenderer + contextBridge
- Python 프로세스와 stdin/stdout JSON 스트리밍 방식으로 통신

3. 데이터 흐름 (Data Flow)

3.1 전체 아키텍처 흐름

```

Renderer (React UI) window.api.xxx() 호출
| ipcRenderer.invoke()
Preload (contextBridge) window.api 객체 노출
| IPC channel
Main Process (index.ts + services)
ipcMain.handle() -> pythonService.runXxx()
| PythonDaemonPool
Worker 1 (stdin/stdout) ... Worker N (stdin/stdout)
|
inference_daemon.py
명령 수신 -> 해당 모듈 호출 -> JSON 응답
캐시: _rcpvms_header_cache / _rcpvms_orbit_cache
|
rcpvms_parser preprocess.py model_loader.py
dmd_parser infer_resnet model_mae.py

```

3.2 AI 추론 데이터 흐름

```

BIN 파일
+--> rcpvms_parser.py (헤더 파싱 + 채널 추출)
+--> preprocess.py
+- extract_rcp_xy_from_bin() <- X/Y 변위 채널
+- estimate_1x_freq() <- 1X 회전 주파수
+- _make_display_pil() <- 궤도 이미지
| [filter_mode: raw / 1x / 2x / broadband / overlay]
infer_resnet_None.py + train_1d_cnn.py
(ResNet18 멀티스케일) (OrbitCNN1D)
| probabilities | probabilities
+----- 양상불 가중합 -----+
|
OOD 판정 (TV Distance + max_conf)
|
normal / abnormal_typeA~C / unknown_abnormal
|
GradCAM + Integrated Gradients 시각화

```

3.3 MAE 이상 탐지 흐름

```

BIN 파일
+--> Stage 1: 슬라이딩 윈도우 배치 스윙
-> 재구성 오차 점수 계산 (1D + Spectral)
-> 최고 점수 윈도우 선정
+--> Stage 2: 선정 윈도우 재평가 (n_eval=10)

```

-> OR 로직: (score_1d > threshold_1d) OR (score_spec > threshold_spec)

-> 최종 판정: normal / abnormal

3.4 궤도 뷰어 데이터 흐름

BIN 파일 선택

+-> rcpvms-info 파일 메타 (채널 수, 샘플링 주파수, 총 시간)

+-> rcpvms-orbit 모든 RCP/시간 윈도우 그리드 정보

+-> rcpvms-orbit-multi (배치 API)

IntersectionObserver: 뷰포트 진입 시만 요청

썸네일 96px (PIL _draw_crosshair, matplotlib 생략)

모달 256px (render_with_axes — 축 레이블 포함)

4. 기능 개요도 (Component Map)

```

App.jsx (탭 라우터)
+-- [양상블 분석] ModelInference.jsx
| +-- AnalysisModeLayout.jsx (공통 레이아웃)
| | +-- SingleFileMode.jsx
| | +-- BatchFileList.jsx
| | +-- BatchProgressBar.jsx
| | +-- BatchActionButtons.jsx
| | +-- BatchResultList.jsx
| | +-- ConcurrencySelector.jsx
| | +-- ErrorDisplay.jsx
| +-- labelStrategies.jsx (ensemble 판정 렌더링)
|
+-- [MAE 분석] MAEAnalysis.jsx
| +-- AnalysisModeLayout.jsx (동일 공유)
| +-- labelStrategies.jsx (mae 판정 렌더링)
|
+-- [DMD 분석] DmdOrbitViewer.jsx
| +-- SubTabNav.jsx
| +-- [서브탭1] DMD 변환: FileOperationFlow.jsx
| +-- [서브탭2] BIN 궤도 뷰어: FileOperationFlow.jsx -> OrbitGrid.jsx
|
+-- [RCPVMS 뷰어] RcpvmsOrbitViewer.jsx
+-- FileOperationFlow.jsx
+-- OrbitControls.jsx (필터 모드 / window_sec / axis_lim)
+-- OrbitGrid.jsx (IntersectionObserver + 배치 API)

```

5. 프로그램 다운로드 및 환경 세팅

5.1 사전 준비

소프트웨어	권장 버전	비고
Git	최신	https://git-scm.com
Node.js	20.x LTS 이상	https://nodejs.org
Python	3.10 이상 (3.12 권장)	"Add to PATH" 필수 체크
Visual Studio Code	최신	선택 사항

5.2 코드 다운로드

```
cd D:\projects
git clone https://github.com/RealGain-5/BE.git rcp_5th
cd rcp_5th
```

5.3 Node.js 의존성 설치

```
npm install
cd src/renderer && npm install && cd ../..
```

※ npm install 후 postinstall 스크립트가 better-sqlite3를 Electron 버전에 맞게 자동 재빌드합니다.

※ 오류 발생 시 수동 실행: npx @electron/rebuild

5.4 Python 가상환경 세팅

```
.Wenv\Scripts\Activate.ps1
# PowerShell 실행 정책 오류 시:
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
python -m pip install --upgrade pip
pip install -r python/requirements.txt
```

※ GPU 사용 시: pip install torch torchvision --index-url https://download.pytorch.org/whl/cu121

5.5 AI 모델 파일 배치

```
python/model/
+-- resnet18_orbit_multiscale.pth 앙상블 ResNet18
+-- orbit_cnn1d.pth 앙상블 CNN1D
+-- orbit_mae.pth MAE 이상 탐지
```

※ 모델 파일이 없으면 담당자에게 요청하십시오.

5.6 설정 파일 확인

```
python/ensemble_config.json 앙상블 가중치 / OOD 임계값
python/mae_config.json MAE 임계값
ensemble_config.json 기본값:
```

```
{ "resnet_weight": 0.5, "cnn1d_weight": 0.5,  
  "ood_threshold": 0.70, "tv_threshold": 0.30 }
```


6. 개발 모드 실행 방법

6.1 개발 모드 실행

```
npm run dev
```

내부 동작 순서:

- Vite 개발 서버(렌더러) 시작
- Electron 메인 프로세스 시작
- BrowserWindow 생성 → 렌더러 URL 로드
- Python 데몬 자동 시작: `venv/Scripts/python python/inference_daemon.py`

6.2 개발 모드 확인 포인트

확인 사항	방법
Python 데몬 준비 확인	콘솔에 'model loaded successfully' 출력 대기
DevTools 열기	F12 키 (개발 모드에서만 활성화)
메인 프로세스 로그	Electron 터미널 창 확인
렌더러 로그	DevTools Console 탭 확인

6.3 TypeScript 타입 체크

```
npm run typecheck
```

7. 프로덕션 빌드 방법

```
# Windows NSIS 설치 파일 빌드
```

```
npm run build:win
```

```
# 언팩 빌드 (테스트용)
```

```
npm run build:unpack
```

빌드 결과물: `dist/` 폴더에 `.exe` 설치 파일 생성

8. 주요 파일 위치 빠른 참조

목적	파일 경로
탭 추가/수정	src/renderer/src/utlis/tabRegistry.js
IPC 핸들러 추가	src/main/index.ts
Python 서비스 메서드 추가	src/main/services/pythonService.ts
preload API 노출	src/preload/index.ts
Python 명령 처리	python/inference_daemon.py
신호 처리/필터링	python/preprocess.py
AI 모델 정의	python/model_mae.py, python/infer_resnet_None.py
양상블 설정	python/ensemble_config.json

9. 미완료 작업 및 인수인계 체크리스트

항목	상태	비고
양상블 모델 재학습	미완료	OE + 복합 지표 코드 완성, 실제 재학습 필요
일반화 평가	미완료	다른 날짜/조건 BIN 파일 테스트 필요
DMD 실제 파일 변환 검증	미완료	5.1GB DMD 파일 변환 테스트 필요
변환 BIN → 학습 파이프라인 투입	미완료	변환된 BIN 파일로 CNN 재학습 필요